

4×4 脉动阵列矩阵乘法验证环境搭建教程

目录

一、 设计概述	2
二、 测试点分解	2
2.1 什么是测试点分解?	2
2.2 功能导向分解法	2
三、 测试用例设计	5
1. 功能测试用例	5
2. 控制逻辑测试用例	6
3. 时序测试用例	6
4. 错误处理测试用例	6
四、 验证环境框架及组件搭建	7
1. 接口定义(interface.sv)	7
2. 事务定义(transaction.sv)	10
3. 生成器(generator.sv)	12
4. 驱动器(driver.sv)	14
5. 监视器 (monitor.sv)	16
6. 记分板(scoreboard.sv)	17
7. 环境(environment.sv)	17
8. 测试台(testbench.sv)	19

一、设计概述

本教程针对一个 4×4 脉动阵列矩阵乘法单元进行验证环境搭建。该设计特点如下：

输入：两个 4×4 矩阵（A 和 B），通过 a_in0-3 和 b_in0-3 输入

输出：16 个 16 位结果(c_out_00 到 c_out_33)

控制信号：时钟(clk)、复位(rst_n)、启动(start)、输入有效(in_valid)、完成(done)

内部结构：16 个处理单元(PE)组成的脉动阵列

数据流：A 矩阵数据向右传递，B 矩阵数据向下传递

计算特点：乘累加操作，结果在 7 个周期后开始输出

二、测试点分解

2.1 什么是测试点分解？

测试点分解是芯片验证的核心环节，指将芯片功能分解为可验证的最小功能单元（测试点）的过程。通过系统化的分解方法，确保验证覆盖所有设计功能和边界条件。

功能类、性能类、DFx 类、边界值

2.2 功能导向分解法

适用场景：算法类模块（如脉动阵列、DSP）

- 分解维度：
- 核心计算功能
- 数据精度处理
- 特殊运算模式
- 性能指标

示例（矩阵乘法单元）：

plaintext

1. 基本乘法功能

整数矩阵乘法正确性

2. 精度处理

溢出处理机制

3. 特殊运算

零矩阵处理

单位矩阵处理

● 1. 功能正确性

基本功能：验证矩阵乘法功能正确性

边界值：

全 0 矩阵乘法

全 1 矩阵乘法

最大值矩阵(255)乘法

混合边界值测试

特殊矩阵：

单位矩阵乘法

对角矩阵乘法

随机矩阵乘法

测试点划分举例：

输入激励为全 1 矩阵时，输出矩阵检测结果为全 4 矩阵

- 2. 控制逻辑

复位功能验证

启动(start)信号有效性

输入有效(in_valid)信号控制

完成(done)信号生成时机

测试点划分举例：当 in_valid 早于 start 拉高时，实际从 start 开始启动。

- 3. 时序行为

数据输入时序（连续输入、间隔输入）

结果输出时序（延迟、顺序）

完成信号时序（有效时间、持续时间）

- 4. 错误处理

复位中断计算

无效输入处理

控制信号异常

- 5. 性能评估

计算延迟（从开始到完成）

吞吐量评估

三、测试用例设计

1. 功能测试用例

用例 ID	描述	输入 A	输入 B	预期结果
F001	全零矩阵	4x4 全 0	4x4 全 0	所有输出为 0
F002	单位矩阵乘法	单位矩阵	单位矩阵	输出单位矩阵
F003	最大值矩阵	所有元素 255	所有元素 255	所有输出为 $255*255*4=260100$
F004	混合边界值	第一行 255 其余 0	第一列 255 其余 0	仅 $c_out_00=255*255$
F005	随机矩阵 1	随机生成	随机生成	参考模型计算结果
F006	随机矩阵	随机生成	随机生成	参考模型计算结果

用例 ID	描述	输入 A	输入 B	预期结果
	阵 2			

2. 控制逻辑测试用例

用例 ID	描述	测试重点
C001	复位功能	复位后所有输出清零
C002	启动信号延迟	start 在不同时间点激活
C003	输入有效信号间隔	in_valid 中间插入无效周期
C004	连续计算	完成信号后立即启动新计算

3. 时序测试用例

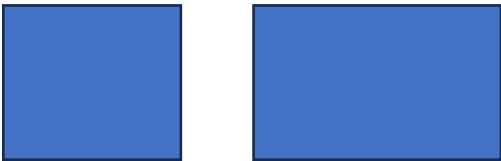
用例 ID	描述	测试重点
T001	输出延迟	验证第一个和最后一个输出时间
T002	完成信号时序	done 在正确周期拉高
T003	背靠背计算	连续两次计算时序验证

4. 错误处理测试用例

用例 ID	描述	测试重点
-------	----	------

用例 ID	描述	测试重点
E001	计算中复位	复位中断当前计算
E002	无效输入处理	in_valid 无效时数据忽略

四、验证环境框架及组件搭建



1. 接口定义(interface.sv)
- 1.1 接口定义的核心作用
- 接口定义在芯片验证中扮演着物理连接抽象化的关键角色，它通过封装设计的所有输入/输出信号，实现：
- **信号统一管理：**集中声明所有设计信号
 - **时序规范定义：**明确信号的时钟域关系
 - **验证复用基础：**解耦验证环境与具体设计实现

```
interface systolic_array_if(input clk);

    logic rst_n;

    logic start;

    logic in_valid;

    logic [7:0] a_in0, a_in1, a_in2, a_in3;

    logic [7:0] b_in0, b_in1, b_in2, b_in3;

    logic [15:0] c_out [4][4];

    logic done;

    // 时钟块

    clocking cb @(posedge clk);

    //驱动

        output rst_n, start, in_valid, a_in0, a_in1, a_in2, a_in3;

        output b_in0, b_in1, b_in2, b_in3;

    //采样

        input done, c_out;

    endclocking

    // 断言

    property reset_prop;

        !rst_n |-> (done == 0) && (c_out[0][0] == 0) && (c_out[3][3]

== 0);
```



```
endproperty

    assert_reset: assert property (reset_pprop);
endinterface
```

2. 事务定义(transaction.sv)

sa_transaction 类的定义在芯片验证环境中扮演着数据建模与抽象的关键角色，它是验证环境的基础数据结构。

- **层次化封装：**将输入、输出和约束整合在单一对象中
- **面向对象设计：**支持继承和多态，便于扩展
- **自包含功能：**集成数据生成、计算和显示功能

```
class sa_transaction;

    // 输入矩阵

    rand bit [7:0] A[4][4];

    rand bit [7:0] B[4][4];


    // 预期输出

    bit [15:0] expected_C[4][4];


    // 约束

    constraint matrix_constraints {

        foreach(A[i,j]) {

            A[i][j] inside {[0:255]};

            B[i][j] inside {[0:255]};

        }

    }

}
```

```
// 参考模型计算

function void compute_expected();

    for (int i = 0; i < 4; i++) begin

        for (int j = 0; j < 4; j++) begin

            expected_C[i][j] = 0;

            for (int k = 0; k < 4; k++) begin

                expected_C[i][j] += A[i][k] * B[k][j];

            end

        end

    end

endfunction

// 显示内容

function void display(string prefix = "");

    $display("%sTransaction: ", prefix);

    $display("A Matrix:");

    foreach(A[i,j]) $write("%4d%c", A[i][j], (j==3)? "\n": " ");

    $display("\nB Matrix:");

    foreach(B[i,j]) $write("%4d%c", B[i][j], (j==3)? "\n": " ");

endfunction

endclass
```

3. 生成器(generator.sv)

```
class generator;

    mailbox gen2drv;

    int num_transactions;

    bit [7:0] special_matrices[4][4][4] = '{
        '{1,0,0,0}, '{0,1,0,0}, '{0,0,1,0}, '{0,0,0,1}},// 单位矩阵
        '{0,0,0,0}, '{0,0,0,0}, '{0,0,0,0}, '{0,0,0,0}},// 全 0
    };

    function new(mailbox gen2drv);
        this.gen2drv = gen2drv;
    endfunction

    task run();
        sa_transaction tr;

        for (int i = 0; i < num_transactions; i++) begin
            tr = new();

            // 特殊矩阵测试

            if (i < 4) begin
                tr.A = special_matrices[i];
                tr.B = special_matrices[i];
            end
        end
    endtask
endclass
```

```
        end

        // 随机测试

        else if (!tr.randomize()) begin

            $error("Generator: Transaction randomization
failed");

            continue;

        end

        tr.compute_expected();

        gen2drv.put(tr);

    end

endtask

endclass
```

4. 驱动器(driver.sv)

```
class driver;
    virtual systolic_array_if vif;
    mailbox gen2drv;
    event driver_done;

    function new(virtual systolic_array_if vif, mailbox gen2drv);
        this.vif = vif;
        this.gen2drv = gen2drv;
    endfunction

    task reset();
        vif.cb.rst_n <= 0;
        vif.cb.start <= 0;
        vif.cb.in_valid <= 0;
        repeat(5) @(vif.cb);
        vif.cb.rst_n <= 1;
        repeat(2) @(vif.cb);
    endtask

    task run();
        forever begin
            sa_transaction tr;
            gen2drv.get(tr);

            // 应用复位
            reset();

            // 启动信号
            vif.cb.start <= 1;
            @(vif.cb);
            vif.cb.start <= 0;

            // 输入数据 (7 个周期)
            for (int cycle = 0; cycle < 7; cycle++) begin
                vif.cb.in_valid <= 1;

                // A 矩阵按列输入, B 矩阵按行输入
                if (cycle < 4) begin
                    vif.cb.a_in0 <= tr.A[0][cycle];
                    vif.cb.a_in1 <= tr.A[1][cycle];
                    vif.cb.a_in2 <= tr.A[2][cycle];
                    vif.cb.a_in3 <= tr.A[3][cycle];
```

```
        vif.cb.b_in0 <= tr.B[cycle][0];
        vif.cb.b_in1 <= tr.B[cycle][1];
        vif.cb.b_in2 <= tr.B[cycle][2];
        vif.cb.b_in3 <= tr.B[cycle][3];
    end else begin
        // 后三个周期输入 0
        vif.cb.a_in0 <= 0;
        vif.cb.a_in1 <= 0;
        vif.cb.a_in2 <= 0;
        vif.cb.a_in3 <= 0;

        vif.cb.b_in0 <= 0;
        vif.cb.b_in1 <= 0;
        vif.cb.b_in2 <= 0;
        vif.cb.b_in3 <= 0;
    end
    @(vif.cb);
end
vif.cb.in_valid <= 0;

// 等待计算完成
while (!vif.cb.done) @(vif.cb);

->driver_done;
end
endtask
endclass
```

5. 监视器 (monitor.sv)

```
class monitor;
    virtual systolic_array_if vif;
    mailbox mon2scb;

    function new(virtual systolic_array_if vif, mailbox mon2scb);
        this.vif = vif;
        this.mon2scb = mon2scb;
    endfunction

    task run();
        forever begin
            // 等待计算完成
            @(posedge vif.done);

            // 收集结果
            sa_transaction tr = new();
            tr.expected_C[0][0] = vif.c_out[0][0];
            tr.expected_C[0][1] = vif.c_out[0][1];
            tr.expected_C[0][2] = vif.c_out[0][2];
            tr.expected_C[0][3] = vif.c_out[0][3];
            tr.expected_C[1][0] = vif.c_out[1][0];
            tr.expected_C[1][1] = vif.c_out[1][1];
            tr.expected_C[1][2] = vif.c_out[1][2];
            tr.expected_C[1][3] = vif.c_out[1][3];
            tr.expected_C[2][0] = vif.c_out[2][0];
            tr.expected_C[2][1] = vif.c_out[2][1];
            tr.expected_C[2][2] = vif.c_out[2][2];
            tr.expected_C[2][3] = vif.c_out[2][3];
            tr.expected_C[3][0] = vif.c_out[3][0];
            tr.expected_C[3][1] = vif.c_out[3][1];
            tr.expected_C[3][2] = vif.c_out[3][2];
            tr.expected_C[3][3] = vif.c_out[3][3];

            mon2scb.put(tr);
        end
    endtask
endclass
```


6. 记分板(scoreboard.sv)

```

class scoreboard;
    mailbox mon2scb;
    int error_count = 0;
    int total_count = 0;

    function new(mailbox mon2scb);
        this.mon2scb = mon2scb;
    endfunction

    task run();
        forever begin
            sa_transaction tr;
            mon2scb.get(tr);
            total_count++;

            // 比较结果
            bit error = 0;
            for (int i = 0; i < 4; i++) begin
                for (int j = 0; j < 4; j++) begin
                    if (tr.expected_C[i][j] != tr.expected_C[i][j]) begin
                        $error("Mismatch at [%0d][%0d]: Exp=%0d, Act=%0d",
                            i, j, tr.expected_C[i][j], tr.expected_C[i][j]);
                        error = 1;
                    end
                end
            end

            if (error) error_count++;
            else $display("Test %0d PASSED", total_count);
        end
    endtask

    function void report();
        $display("\nTEST SUMMARY");
        $display("Total tests : %0d", total_count);
        $display("Passed      : %0d", total_count - error_count);
        $display("Failed      : %0d", error_count);
        $display("Pass rate   : %0.2f%%", (real'(total_count - error_count) / total_count *
100);
    endfunction
endclass

```

7. 环境(environment.sv)

```
class environment;
    generator gen;
    driver drv;
    monitor mon;
    scoreboard scb;

    mailbox gen2drv;
    mailbox mon2scb;

    virtual systolic_array_if vif;

    function new(virtual systolic_array_if vif);
        this.vif = vif;

        // 创建 mailbox
        gen2drv = new();
        mon2scb = new();

        // 创建组件
        gen = new(gen2drv);
        drv = new(vif, gen2drv);
        mon = new(vif, mon2scb);
        scb = new(mon2scb);
    endfunction

    task pre_test();
        $display("Applying reset...");
        drv.reset();
        $display("Reset complete");
    endtask

    task test();
        $display("Starting test");

        fork
            gen.run();
            drv.run();
            mon.run();
            scb.run();
        join_any
    endtask

    task post_test();
        wait(gen.num_transactions > 0 && scb.total_count == gen.num_transactions);
```

```

        scb.report();
        $display("Test completed");
    endtask

    task run();
        pre_test();
        test();
        post_test();
        $finish;
    endtask
endclass

```

8. 测试台(testbench.sv)

```

module tb;
    // 时钟和复位
    bit clk;
    bit rst_n;

    // 时钟生成
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // 接口实例化
    systolic_array_if sa_if(clk);

    // 连接复位
    assign sa_if.rst_n = rst_n;

    // DUT 实例化
    systolic_array dut(
        .clk(clk),
        .rst_n(sa_if.rst_n),
        .start(sa_if.start),
        .in_valid(sa_if.in_valid),
        .a_in0(sa_if.a_in0), .a_in1(sa_if.a_in1), .a_in2(sa_if.a_in2), .a_in3(sa_if.a_in3),
        .b_in0(sa_if.b_in0), .b_in1(sa_if.b_in1), .b_in2(sa_if.b_in2), .b_in3(sa_if.b_in3),
        .c_out_00(sa_if.c_out[0][0]), .c_out_01(sa_if.c_out[0][1]),
        .c_out_02(sa_if.c_out[0][2]), .c_out_03(sa_if.c_out[0][3]),
        .c_out_10(sa_if.c_out[1][0]), .c_out_11(sa_if.c_out[1][1]),
        .c_out_12(sa_if.c_out[1][2]), .c_out_13(sa_if.c_out[1][3]),
        .c_out_20(sa_if.c_out[2][0]), .c_out_21(sa_if.c_out[2][1]),
        .c_out_22(sa_if.c_out[2][2]), .c_out_23(sa_if.c_out[2][3]),
    );
endmodule

```

```
.c_out_30(sa_if.c_out[3][0]), .c_out_31(sa_if.c_out[3][1]),  
.c_out_32(sa_if.c_out[3][2]), .c_out_33(sa_if.c_out[3][3]),  
.done(sa_if.done)  
);  
  
// 环境实例化  
environment env;  
  
initial begin  
    env = new(sa_if);  
    env.gen.num_transactions = 20; // 20 个测试用例  
    env.run();  
end  
  
// 波形记录  
initial begin  
    $dumpfile("waveform.vcd");  
    $dumpvars(0, tb);  
end  
  
// 超时结束  
initial begin  
    #1000000;  
    $display("Simulation timeout");  
    $finish;  
end  
endmodule
```